# Proyecto Final Tecnologías de Desarrollo Web

## Curso 2025 - 2026

Asier Prieto Domínguez

Desarrollo de una Aplicación Web completa integrando los conocimientos adquiridos a lo largo de la asignatura. Esta aplicación web está dividida en dos subsistemas:

**Subsistema Gestor:** gestión completa de todos los recursos del sistema (usuarios, operaciones, operadores y puntos) CRUD completo. Además del resto de funcionalidades del subsistema público

**Subsistema Público:** permite ver el panel informativo, registrarse y gestionar el perfil.

La integración de ambos subsistemas se realiza con la API REST con autenticación JWT.

# 1. Arquitectura del Sistema

Arquitectura cliente-servidor donde el frontend y el backend se comunican exclusivamente a través de la API REST

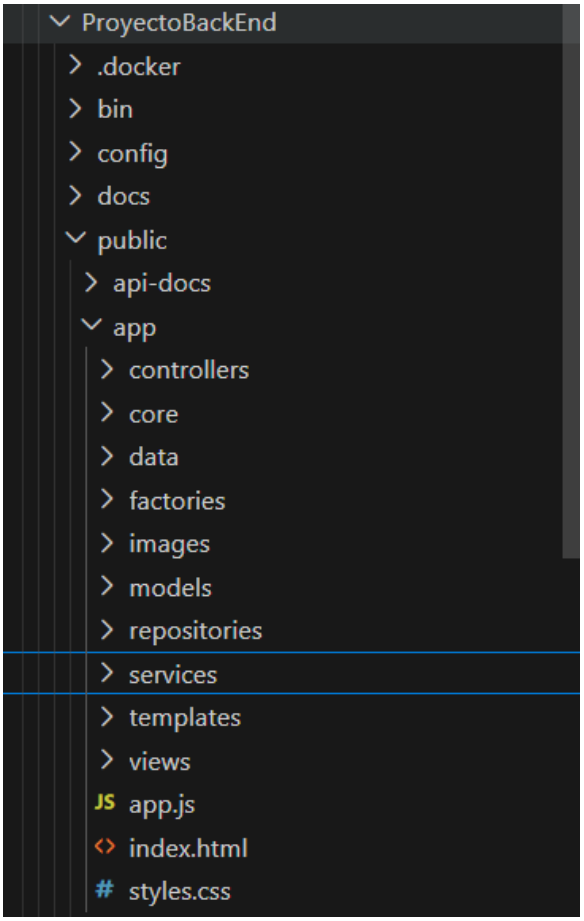
## 1.1 Backend

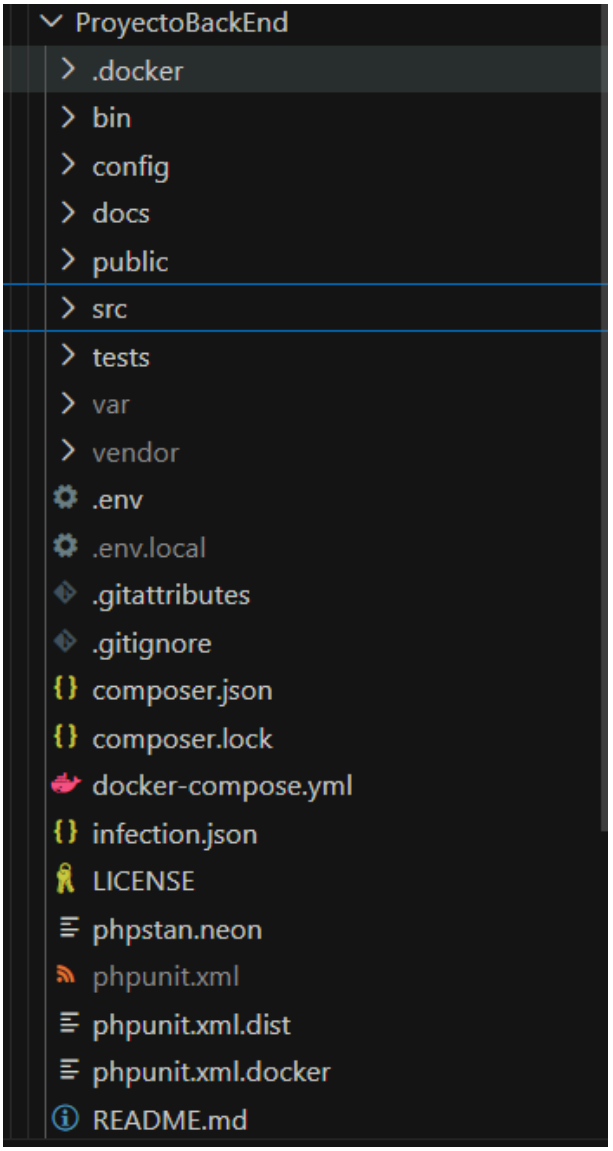
Desarrollado con PHP y el framework Slim4. Usamos Doctrine ORM para el acceso a la base de datos MySQL y la autenticación está implementada mediante JWT

## 1.2 Frontend

Desarrollado con JavaScript siguiendo la arquitectura MVC y la POO

# 2. Estructura del Proyecto



El frontend está en la misma carpeta public del backend para eliminar los problemas de CORS al compartir el mismo origen (http://127.0.0.1:8000)

# 3. Patrones de Diseño Aplicados

## 3.1 Singleton en Session.js

Garantiza la existencia de una única instancia de sesión compartida por toda la aplicación

## 3.2 Repository en el acceso a los datos

Cada entidad tiene su propio repositorio que encapsula el acceso a datos. El controlador nunca hace llamadas HTTP directamente.

La persistencia ha pasado de realizarse con localStorage y los IDs siendo generados por el propio cliente a usar API REST y los IDs ser asignados por el backend de forma segura

## 3.3 Factory

Hemos pasado de usar el factory en el cliente para construir los objetos antes de guardarlos en el localStorage a tener la responsabilidad de construir objetos en el backend a través de los CreateCommand.php de cada recurso

Mejoramos la seguridad al no tener controlado el acceso simultáneo en el primer caso y si tenerlo en el backend que valida los datos antes de persistirlos y devuelve el objeto ya construido

# 4. Especificación de la API REST

## 4.1 Autenticación

Basada en JWT, el token se obtiene en el endpoint de login y debe incluirse en todas las peticiones protegidas como cabecera HTTP

## 4.2 Endpoints

**Autenticación**

POST /access_token

**Usuarios**

GET /users

GET /users/{id}

GET /users/email/{email}

POST /users

PUT /users/{id}

DELETE /users/{id}

**Operadores**

GET /operators

POST /operators

PUT /operators/{id}

DELETE /operators/{id}

**Puntos**

GET /spots

POST /spots

PUT /spots/{id}

DELETE /spots/{id}

**Operaciones**

GET /operations

POST /operations

PUT  /operations/{id}

DELETE  /operations/{id}

## 4.3 Gestión del ETag

El backend implementa el locking mediante ETag para las operaciones PUT y DELETE requieren la cabecera If-Match con el ETag del recurso, si no coincide se devuelve error 428

# 5. Ampliación del Modelo User

Se añaden los campos de perfil al modelo User.php original según se especificaba en el enunciado:

- Nombre
- Apellidos
- Fecha de Nacimiento
- Teléfono (adicional)
- URLs de Interés

# 6. Funcionalidades Implementadas

## 6.1 Subsistema Gestor

Gestión completa de Operaciones, crear con todos los campos, editar estado, hora estimada, operador y punto y eliminar la operación

Gestión completa de Operadores, crear operador con todos los campos, editar nombre, siglas, color y URL icono y eliminar el operador

Gestión completa de Puntos, crear puntos con todos los campos, editar y eliminar el punto

Gestión completa de Usuarios, cambiar el rol, activar desactivar usuario, eliminar

## 6.2 Subsistema Público

Panel Informativo con filtro por SALIDA/LLEGADA, estado y búsqueda de la operación por código

Ordenación del panel informativo por hora, código, destino/origen y operador

Registro de nuevos usuarios con comprobación de email por AJAX en tiempo real

Vista de perfil para editar los datos personales del usuario activo

## 6.3 Comprobación de email por AJAX

El formulario de registro comprueba en tiempo real si el email introducido está ya registrado llamando al endpoint GET /users/email/{email} sin autenticación.

# 7. Mecanismos de Protección

## 7.1 Autenticación JWT

Todas las rutas protegidas requieren JWT en la cabecera de Autorización, el JWT se almacena en sessionStorage por seguridad y se elimina al cerrar el navegador o la pestaña

## 7.2 Autorización

Comprobación del rol antes de ejecutar operaciones restringidas o renderizar secciones protegidas

## 7.3 Contraseñas

Almacenadas con password_hash de PHP, no se guardan ni transmiten en texto plano

## 7.4 Usuarios Inactivos

Se conservan los datos del usuario pero se prohíbe el login

## 7.5 ETag

Implementado el control de concurrencia optimista mediante ETag en las operaciones PUT y DELETE

# 8. Diseño CSS Responsive

Uso de variables :root para un diseño consistente en toda la aplicación

Uso de Flexbox para la navegación principal, los formularios, botones y la barra de admin

Uso de CSS Grid en el panel de controles del tablero informativo

Uso de Media Queries para formularios compactos, botones en columna, nav con scroll horizontal en móvil, con ancho máximo de 600px nav en varias líneas controles del filtro en una sola columna con ancho máximo de 900px y mayor padding en contenedores y nav con más espacio con ancho mínimo de 1400px

# 9. Problemas y Soluciones Propuestas

CORS con servidor PHP buitl-in, el DELETE Y PUT quedaban bloqueados por preflight OPTIONS, se sirve el frontend desde /public/app del backend con el mismo origen el CORS queda eliminado

Backend devuelve 404 cuando un repositorio está vacío, los repositorios tratan ese 404 cómo warning en vez de error para no bloquear la aplicación, ocurre cuando no se carnage repositorio de maneras innecesaria, como cuando no inicias sesión el repositorio de usuarios

Input datetime-local daba problema al devolver un formato que no aceptaba MySQL y se guardaba como null las fechas cuando se creaba/editaba una operación, se realiza una conversión en el frontend que soluciona este problema

Arreglo de la clase app.js dividiéndola en varias clases Router.js, Application.js y SessionManager.js para evitar que app.js sea una god class y rompiese con la POO

## 10. Conclusiones

Se han integrado todos los conocimientos adquiridos durante la asignatura en este proyecto

El sistema resultante cumple con todos los requisitos funcionales establecidos en el enunciado. Además de pasar los tests que estaban desde la práctica anterior del backend